In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
```

In [3]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

In [4]:
```
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
```

In [5]:
```
from sklearn.inspection import permutation_importance
```

In [6]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [7]:
```
df = pd.read_csv('original.csv')
```

In [8]:
```
print("Ukuran Dataset:", df.shape)
```

```
Ukuran Dataset: (87, 22)
```

In [9]:
```
display(df.head())
```

	Gender	Age	POLAR4 Quintile	POLAR3 Quintile	Adult HE 2001 Quintile	Adult HE 2011 Quintile	TUNDRA MSOA Quintile	TUNDRA LSOA Quintile	Gaps GCSE Quintile	Gap GCS Ethnici Quinti
0	0.0	38.0	5.0	5.0	4.0	5.0	4.0	4.0	5.0	5
1	0.0	22.0	4.0	2.0	1.0	3.0	4.0	4.0	4.0	5
2	0.0	21.0	4.0	5.0	4.0	5.0	4.0	4.0	5.0	5
3	0.0	25.0	5.0	4.0	3.0	4.0	3.0	3.0	5.0	5
4	0.0	20.0	5.0	4.0	3.0	4.0	3.0	3.0	4.0	4

5 rows × 22 columns

In [10]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 87 entries, 0 to 86
Data columns (total 22 columns):
 #   Column                            Non-Null Count  Dtype  
---  ------                            --------------  -----  
 0   Gender                            80 non-null     float64
 1   Age                               80 non-null     float64
 2   POLAR4 Quintile                   80 non-null     float64
 3   POLAR3 Quintile                   80 non-null     float64
 4   Adult HE 2001 Quintile            80 non-null     float64
 5   Adult HE 2011 Quintile            80 non-null     float64
 6   TUNDRA MSOA Quintile              80 non-null     float64
 7   TUNDRA LSOA Quintile              80 non-null     float64
 8   Gaps GCSE Quintile                80 non-null     float64
 9   Gaps GCSE Ethnicity Quintile      80 non-null     float64
 10  Uni Connect target ward           80 non-null     float64
 11  attending from home?              80 non-null     float64
 12  distance to university (km)       80 non-null     float64
 13  Count of Module Area Logins       80 non-null     float64
 14  Total Hours in Module Area        80 non-null     float64
 15  % of Average Hours in Module Area 80 non-null     float64
 16  # of presence                     80 non-null     float64
 17  # of Absence                      78 non-null     float64
 18  Percent Attended                  80 non-null     float64
 19  label (fail=1, pass=0)            80 non-null     float64
 20  Unnamed: 20                       0 non-null      float64
 21  Unnamed: 21                       0 non-null      float64
dtypes: float64(22)
memory usage: 15.1 KB
```

In [11]:
```
print("\nData Kosong:")
print(df.isnull().sum())
```

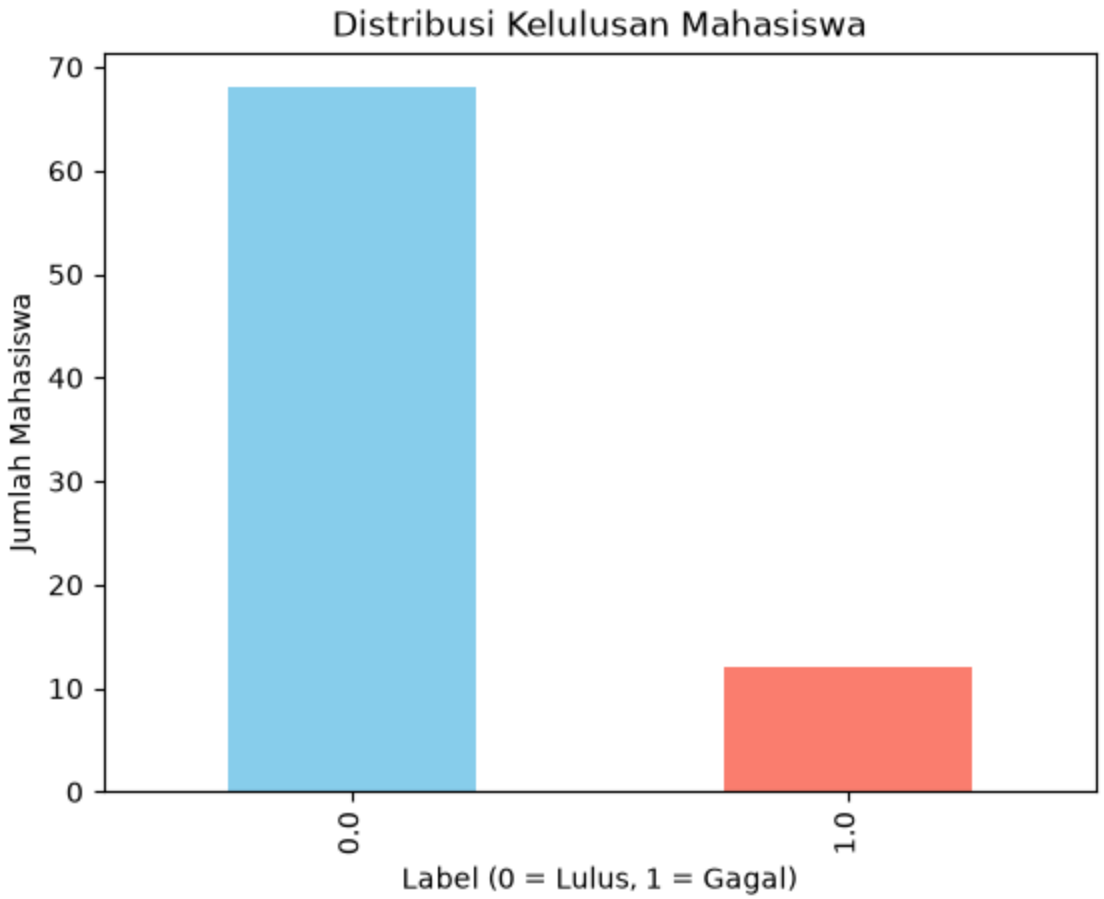
```
Data Kosong:
Gender                                7
Age                                   7
POLAR4 Quintile                       7
POLAR3 Quintile                       7
Adult HE 2001 Quintile                7
Adult HE 2011 Quintile                7
TUNDRA MSOA Quintile                  7
TUNDRA LSOA Quintile                  7
Gaps GCSE Quintile                    7
Gaps GCSE Ethnicity Quintile          7
Uni Connect target ward               7
attending from home?                  7
distance to university (km)           7
Count of Module Area Logins           7
Total Hours in Module Area            7
% of Average Hours in Module Area     7
# of presence                         7
# of Absence                          9
Percent Attended                      7
label (fail=1, pass=0)                7
Unnamed: 20                          87
Unnamed: 21                          87
dtype: int64
```

In [12]:
```
print(df['label (fail=1, pass=0)'].value_counts())
```

```
label (fail=1, pass=0)
0.0    68
1.0    12
Name: count, dtype: int64
```

In [13]:
```
df['label (fail=1, pass=0)'].value_counts().plot(
    kind='bar',
    color=['skyblue', 'salmon']
)

plt.title('Distribusi Kelulusan Mahasiswa')
plt.xlabel('Label (0 = Lulus, 1 = Gagal)')
plt.ylabel('Jumlah Mahasiswa')
plt.show()
```



EDA DAN PEMROSESAN DATA

In [14]:
```
df = df.dropna(how='all')
```

In [15]:
```
df = df.dropna(axis=1, how='all')
```

In [16]:
```
df.columns = df.columns.str.strip()
```

In [17]:
```
df = df.dropna(subset=['label (fail=1, pass=0)'])
```

In [18]:
```
df = df.drop_duplicates()
```

In [19]:
```
print("Ukuran Data Setelah Dibersihkan:", df.shape)
```

```
Ukuran Data Setelah Dibersihkan: (80, 20)
```

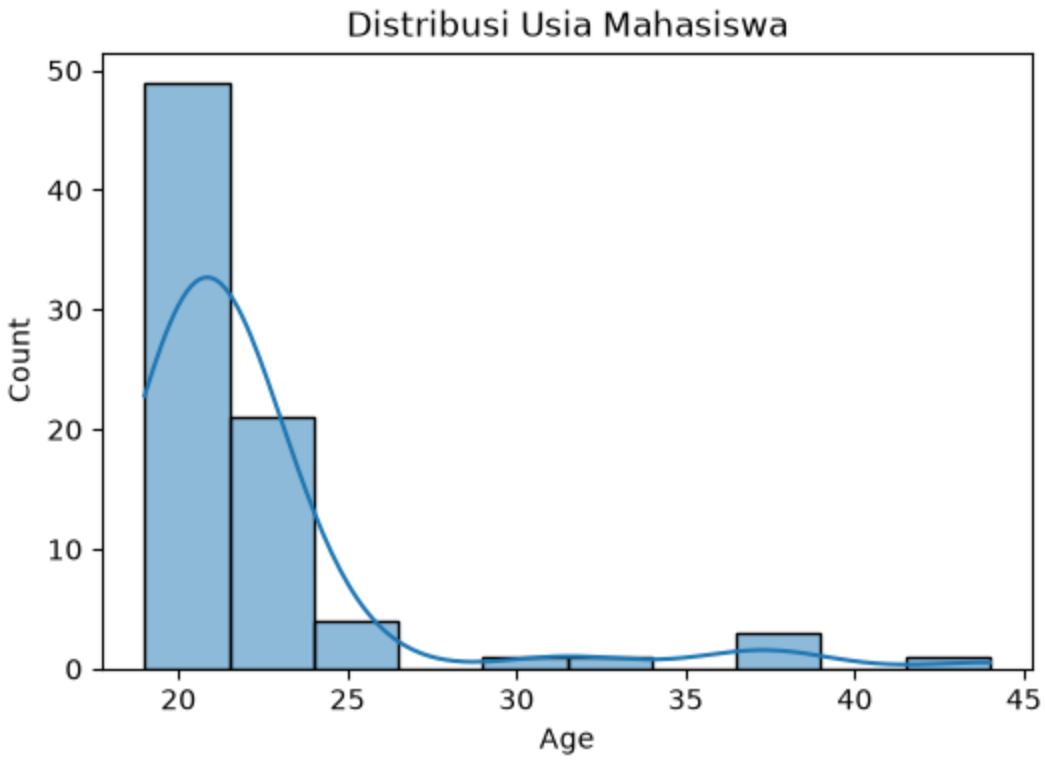
In [20]:
```
kolom_kuintil = [kolom for kolom in df.columns if 'Quintile' in kolom]
```

In [21]:
```
for kolom in kolom_kuintil:
    df[kolom] = df[kolom].replace(0, np.nan)

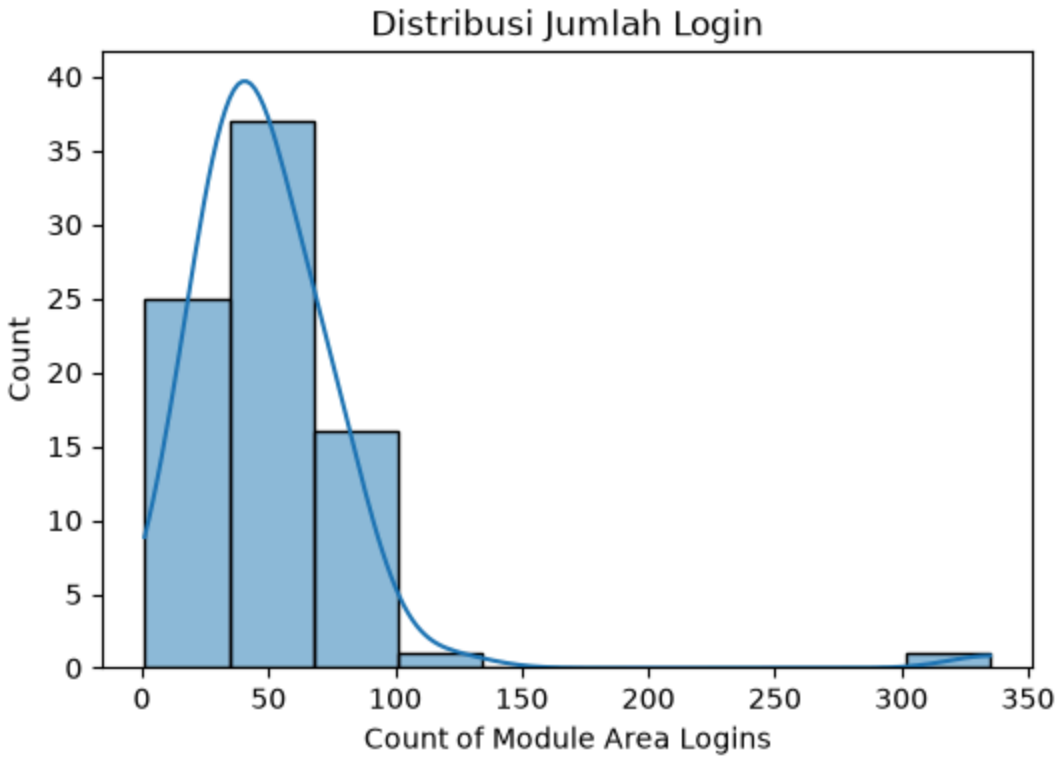
print("Data kosong setelah pemeriksaan:")
print(df.isnull().sum())
```

```
Data kosong setelah pemeriksaan:
Gender                                0
Age                                   0
POLAR4 Quintile                       0
POLAR3 Quintile                       0
Adult HE 2001 Quintile                0
Adult HE 2011 Quintile                2
TUNDRA MSOA Quintile                  2
TUNDRA LSOA Quintile                  2
Gaps GCSE Quintile                    4
Gaps GCSE Ethnicity Quintile          4
Uni Connect target ward               0
attending from home?                  0
distance to university (km)           0
Count of Module Area Logins           0
Total Hours in Module Area            0
% of Average Hours in Module Area     0
# of presence                         0
# of Absence                          2
Percent Attended                      0
label (fail=1, pass=0)                0
dtype: int64
```

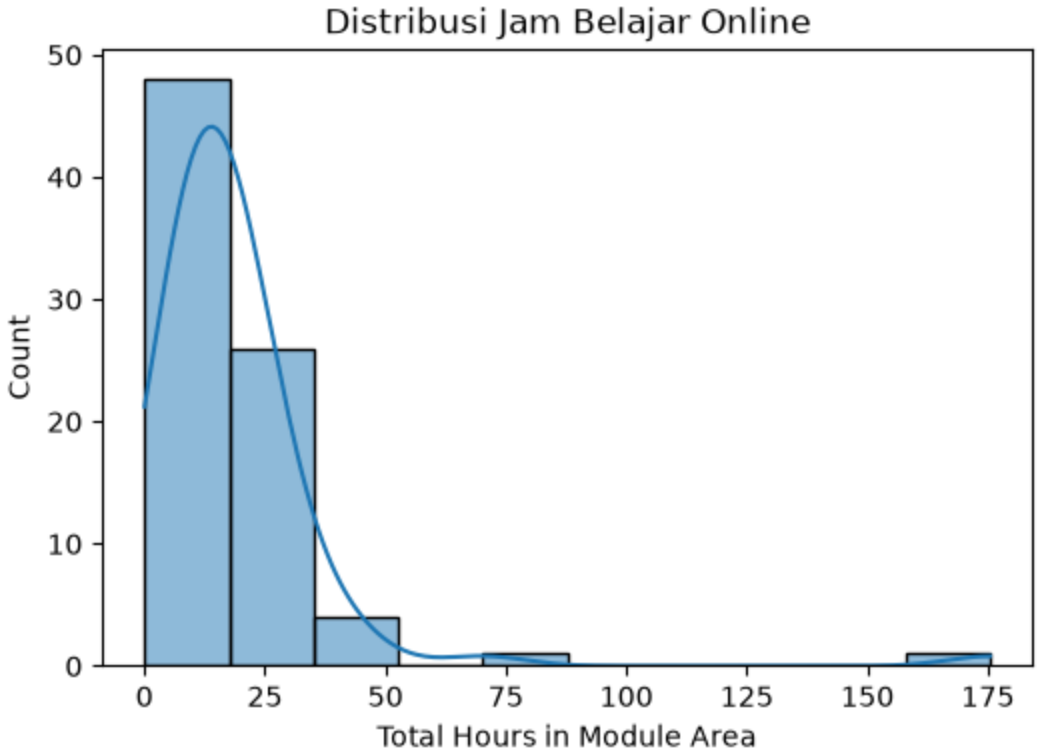
In [22]:
```
plt.figure(figsize=(6,4))
sns.histplot(df['Age'], bins=10, kde=True)
plt.title('Distribusi Usia Mahasiswa')
plt.show()
```



In [23]:
```
plt.figure(figsize=(6,4))
sns.histplot(df['Count of Module Area Logins'], bins=10, kde=True)
plt.title('Distribusi Jumlah Login')
plt.show()
```



In [24]:
```
plt.figure(figsize=(6,4))
sns.histplot(df['Total Hours in Module Area'], bins=10, kde=True)
plt.title('Distribusi Jam Belajar Online')
plt.show()
```



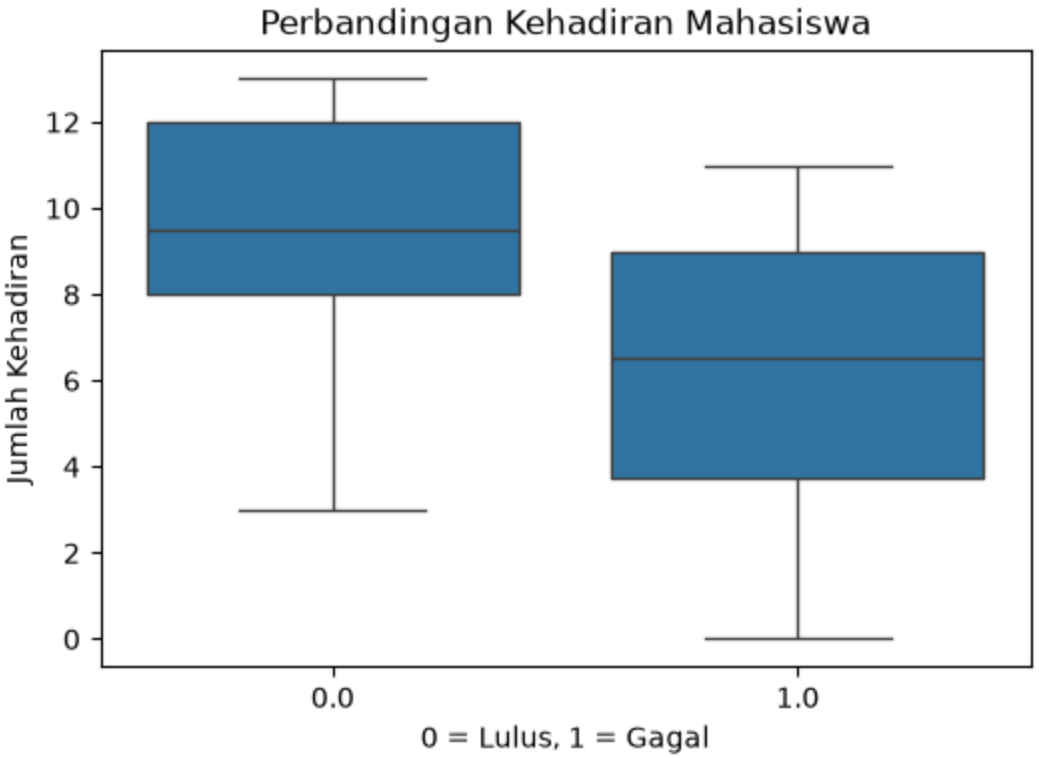
In [25]:
```
plt.figure(figsize=(6,4))

sns.boxplot(
```

```
    x='label (fail=1, pass=0)',
    y='# of presence',
    data=df
)

plt.title('Perbandingan Kehadiran Mahasiswa')
plt.xlabel('0 = Lulus, 1 = Gagal')
plt.ylabel('Jumlah Kehadiran')

plt.show()
```

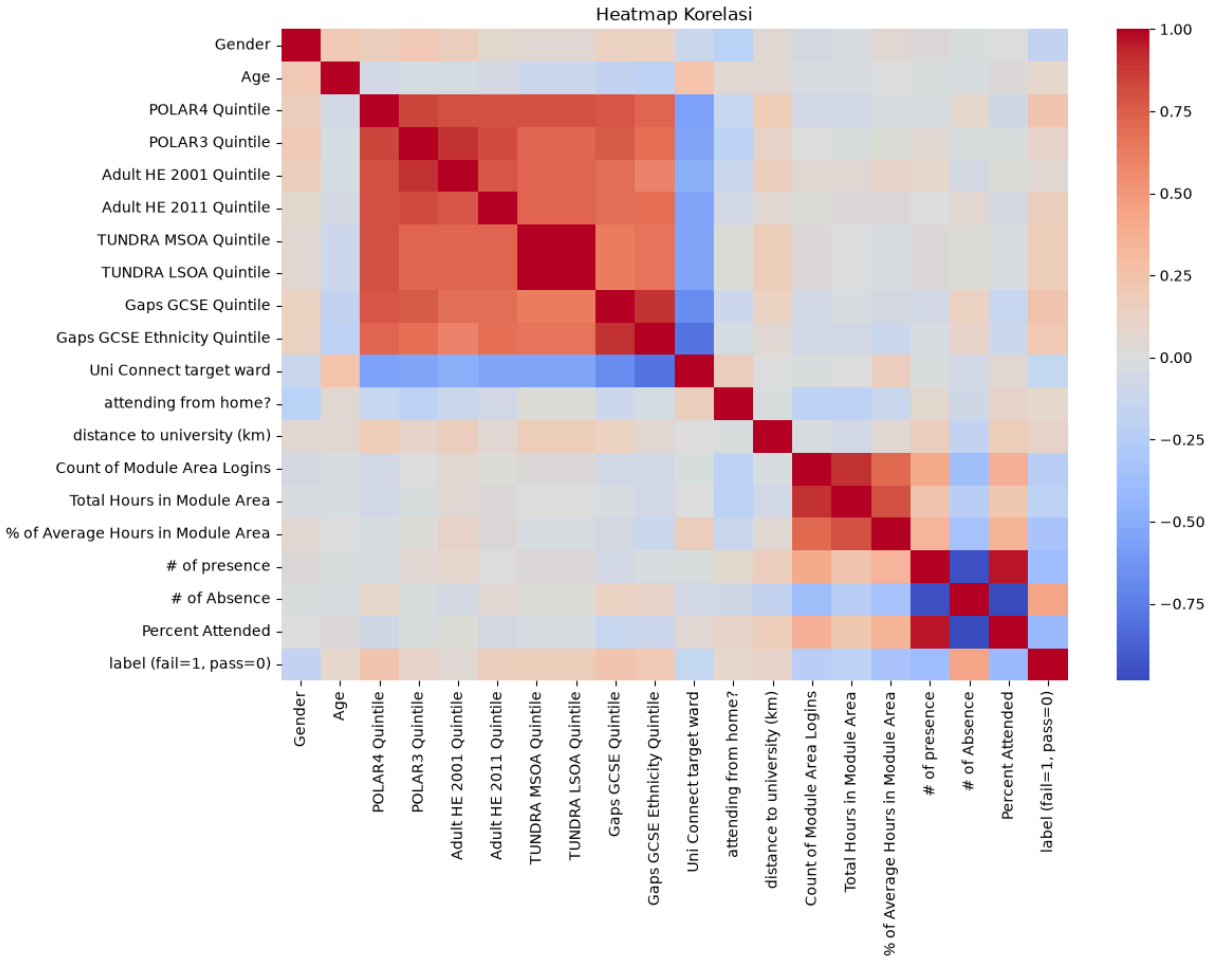


In [26]:
```
plt.figure(figsize=(12,8))

korelasi = df.select_dtypes(include='number').corr()

sns.heatmap(
    korelasi,
    annot=False,
    cmap='coolwarm'
)

plt.title('Heatmap Korelasi')
plt.show()
```



PERSIAPAN DATA

In [27]:
```
df = df.drop(columns=[
    'TUNDRA LSOA Quintile',
    '# of Absence',
    'Percent Attended'
])
```

In [28]:
```
X = df.drop(columns=['label (fail=1, pass=0)'])

y = df['label (fail=1, pass=0)']

print("Jumlah Fitur:", X.shape[1])
print("Jumlah Label:", y.shape[0])
```

```
Jumlah Fitur: 16
Jumlah Label: 80
```

In [29]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)

print("Data Latih:", X_train.shape)
print("Data Uji:", X_test.shape)
```

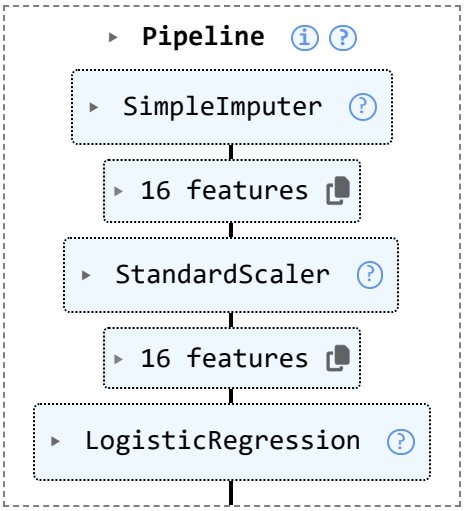
```
Data Latih: (64, 16)
Data Uji: (16, 16)
```

Modelling

In [30]:
```
model_lr = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(
        class_weight='balanced',
        max_iter=2000,
        random_state=42
    ))
])
```

In [31]:
```
model_lr.fit(X_train, y_train)
```

Out[31]:



In [32]:
```
pred_lr = model_lr.predict(X_test)

print("Logistic Regression Berhasil Dilatih")
```

```
Logistic Regression Berhasil Dilatih
```

In [33]:
```
model_rf = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('model', RandomForestClassifier(
        class_weight='balanced',
        random_state=42
    ))
])
```

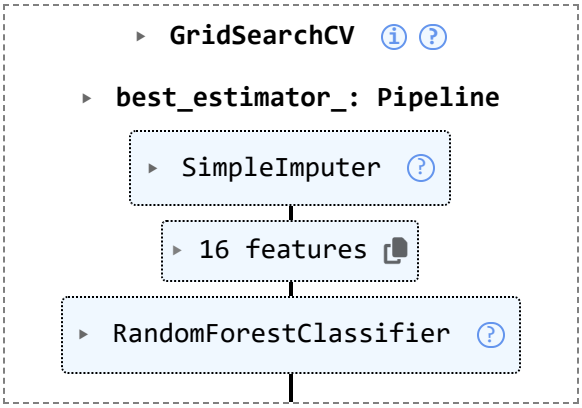
In [34]:
```
parameter = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [None, 3, 5],
    'model__min_samples_leaf': [1, 2]
}
```

In [35]:
```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

In [36]:
```
grid = GridSearchCV(
    model_rf,
    parameter,
    scoring='f1',
    cv=cv,
    n_jobs=-1
)
```

In [37]:
```
grid.fit(X_train, y_train)
```

Out[37]:



In [38]:
```
best_rf = grid.best_estimator_

print("Parameter Terbaik:", grid.best_params_)
```

```
Parameter Terbaik: {'model__max_depth': None, 'model__min_samples_leaf': 1, 'model__
n_estimators': 100}
```

In [ ]:
```

```